

# Multiplicación de Matrices Mediante Paralelismo con MPI

**Paulina Suden, Tomás Rando**

Proyecto Final Integrador de la asignatura Programación Paralela y Distribuida (FIng, UNCuyo)

*Abstract: Se presenta el informe del proyecto realizado para la materia “Programación Paralela y Distribuida” del año 2025. Se realizó un programa paralelo utilizando MPI para resolver el cálculo de matrices. Además, mediante índices, se compara el funcionamiento con respecto a la versión secuencial, también implementada.*

*Keywords: Multiplicación de matrices, paralelo, MPI*

## 1. Introducción

El problema que se resolvió fue la multiplicación de matrices. Para ello, se consideraron matrices de cualquier tamaño, es decir, no sólo cuadradas. Como se menciona en el libro “Linear Algebra and Its Applications”, para la multiplicación de matrices, se necesitan dos de estas. Entonces, si consideramos  $A \in \mathbb{R}^{m \times n}$  y  $B \in \mathbb{R}^{n \times p}$ , al multiplicar ambas, es decir,  $A \times B$ , obtendremos una nueva matriz  $C \in \mathbb{R}^{m \times p}$  cuya entrada ubicada en la posición  $(i, j)$  vendrá dada por:

$$\sum_{k=1}^n A_{i,k} * B_{k,j}$$

En el caso del presente proyecto, se aplicó un procedimiento para poder resolver el problema de manera paralela con el objetivo de aumentar la velocidad del cómputo.

El presente informe se divide en diferentes secciones:

- Desarrollo: Se documentan las decisiones algorítmicas y de diseño tomadas.
- Diseño de experimentos: Se detalla el diferente número de experimentos con sus correspondientes parámetros. Además, se listan los índices tenidos en cuenta para evaluar el desempeño de la implementación.
- Resultados obtenidos: Se exponen los resultados obtenidos de los experimentos correspondientes a la sección anterior.
- Análisis de resultados: Se evalúa y valora el rendimiento y la ganancia obtenidos con la implementación paralela del problema.
- Conclusiones: Se presentan las conclusiones obtenidas a partir de los resultados obtenidos y del análisis realizado sobre los mismos. Además, se indican posibles mejoras.

- Referencias bibliográficas: Se indican las fuentes utilizadas para la realización del trabajo.

## 2. Desarrollo

Las diversas implementaciones realizadas fueron desarrolladas utilizando el lenguaje de programación C++. Además, se utilizó una implementación abierta de MPI, OpenMPI, cuya documentación se incluye en la sección de referencias.

Para poder evaluar el desempeño de la versión paralela se realizó adicionalmente la versión secuencial del programa, sin el agregado de las operaciones propias de los desarrollos paralelos.

### 2.1. Versión Secuencial

Se realizó una implementación secuencial de la solución al problema de la multiplicación de matrices para poder tener una comparación justa a la hora de elaborar los índices y para poder realizar una correcta evaluación del desempeño de la versión paralela.

Para esta implementación, el algoritmo elaborado tiene una complejidad de  $O(n^3)$ , propia de las multiplicaciones de matrices. Además, se utilizó memoria dinámica para poder realizar cálculos con matrices de mayor tamaño, esto debido a que si se utilizaba memoria estática, se podía provocar un desborde del stack. En esta versión, a causa de que se desarrolló con el objetivo de ser ejecutado en 1 procesador, no se incluyó ninguna división de las matrices correspondientes.

A continuación, se muestra un pseudocódigo simplificado de dicha versión:

```
//Entrada: ./matrixSequential n m n1 m1
INICIO
LEER n, m, n1, m1 desde los argumentos de línea de
comandos

SI m != n1 ENTONCES
    mostrar error
FIN SI

CREAR matriz1 de tamaño n x m
CREAR matriz2 de tamaño n1 x m1
CREAR matrizResultado de tamaño n x m1

matrizResultado = MULTIPLICAR matriz1 y matriz2
FIN
```

## 2.2. Versión Paralela

Se tuvieron en cuenta múltiples modelos de programación y diferentes métodos de descomposición del problema. Al principio se pensó en un modelo master-worker con asignación dinámica bajo petición, sin embargo, este modelo podría ser poco escalable, ya que si se agregan nodos o el tamaño de las matrices aumenta, el master se podría saturar a la hora de realizar las comunicaciones y el desempeño en general se vería degradado.

A causa de esto, se optó por un modelo master-worker, pero con asignación estática, es decir, distribuyendo todas las filas al inicio, antes de comenzar la ejecución. Además, esto se implementó junto a una descomposición de dominio ya que este método es más compatible con la multiplicación de matrices.

La descomposición fue aplicada dividiendo las filas de la matriz A según la cantidad de procesos. Es decir, a cada proceso le tocan  $\lfloor \text{filas de } A \div \text{procesos totales} \rfloor$  filas de A, y, en el caso de que existiese un resto pendiente, estas filas sobrantes se reparten entre los primeros procesos. Entonces, por ejemplo, si el resto es 2, el proceso 0 y 1 reciben una fila adicional. Con respecto a la matriz B, esta se transmitió en su totalidad debido a que se necesitaba el uso de esta para calcular cada fila.

Luego de que todos los procesos calculen la multiplicación correspondiente, estos devuelven al proceso 0 sus resultados parciales y este último los une, obteniendo así la matriz resultante. Para realizar todo este procedimiento se hizo uso de comunicación a través de paso de mensajes con MPI. Por ello, para esta implementación en concreto, se utilizaron tres operaciones de comunicación colectiva bloqueante, el *gather*, el *scatter* y el *broadcast*. Esto debido a que se necesitaba que el proceso 0 distribuyese las filas de la matriz A y enviase la matriz B a todos los otros procesos en el mismo momento de la ejecución. Con respecto a los comunicadores utilizados, solo se hizo uso del comunicador global (*COMM\_WORLD*), esto ya que no fue necesaria la división de procesos en diferentes comunicadores, pues todos debían comunicarse con el master.

Con respecto a la sincronización, sólo existe la inherente a las comunicaciones bloqueantes utilizadas. Y, en cuanto a dependencias, el proceso 0 depende de que los demás envíen sus resultados, mientras que el resto dependen de que el proceso 0 les comunique los datos para poder realizar el cómputo.

En conclusión, en esta implementación todos los procesos realizan las mismas tareas: calcular la parte de la matriz resultado que les corresponde. La única excepción es el proceso 0, que además se encarga de calcular el vector de desplazamientos y el de la cantidad de datos que **recibe** por parte de cada proceso en el *gather*, así como de inicializar las matrices A, B y resultado.

En cuanto a la sección secuencial del programa, podemos mencionar estas últimas, junto al cálculo de los vectores de desplazamientos y de conteo de **envíos**, utilizados por la llamada a *scatter*, *ya que aunque el cálculo lo realizan todos los procesos, el resultado es el mismo para todos*. Con respecto a la sección paralelizable, esta se trata de la recepción de los datos, la multiplicación de la porción respectiva y el posterior envío del resultado.

A continuación se muestra un pseudocódigo de la versión paralela simplificada. Diferenciando entre el proceso 0 y el resto:

```
//Entrada: ./matrixParallel n m n1 m1
```

**INICIO**

**LEER** FILAS Y COLUMNAS n, m, n1, m1 de matriz A y matriz B respectivamente

**CALCULAR** las filas que le tocan a cada proceso

**SI** soy el **proceso 0** ENTONCES

**CREAR** e **INICIALIZAR** matrices A y B

**DISTRIBUIR** las filas correspondientes de A a cada proceso (incluido el mismo) usando `MPI\_Scatterv`

**SINO**

**RECIBIR** filas con `MPI\_Scatterv`

**FIN SI**

TODOS los procesos:

**RECIBIR** matriz B completa desde el proceso 0 mediante `MPI\_Bcast`

**MULTIPLICAR** las filas recibidas de matrizA \* matrizB y **GUARDAR** su resultado parcial

**SI** soy el proceso 0 ENTONCES

**CALCULAR** cantidad de datos que recibe de cada proceso

**RECOLECTAR** resultados parciales desde todos los procesos usando `MPI\_Gatherv` y

**RECONSTRUIR** la matriz final

**SINO**

```
        ENVIAR resultados parciales con  
        `MPI_Gatherv` al proceso 0  
FIN SI
```

### 2.3. Parámetros de funcionamiento

Para la ejecución y compilación de la versión secuencial se hará uso de un compilador de C++, en este caso, G++, el cual es parte de GCC. Luego de haber asegurado la instalación de este, se puede proceder a compilar el programa colocando el siguiente comando, dentro de la carpeta que posea al archivo `matrixSequential.cpp`:

```
g++ matrixSequential.cpp -o matrixSequential
```

Luego, se puede proceder a la ejecución del mismo, colocando el siguiente comando:

```
./matrixSequential  
(filasA) (columnasA) (filasB) (columnas B)
```

Para la versión paralela se utilizará el compilador `mpic++`. Se procede a compilar con el siguiente comando (en la ubicación correspondiente):

```
mpic++ matrixParallel.cpp -o matrixParallel
```

Y se puede ejecutar con el comando:

```
mpirun -np (procesos) matrixParallel (filasA)  
(columnasA) (filasB) (columnasB)
```

## 3. Diseño de experimentos

Para los experimentos se tomaron en cuenta diferentes tamaños de matrices:

- Matriz A = 300 x 300 y Matriz B = 300 x 300
- Matriz A = 1000 x 1000 y Matriz B = 1000 x 1000
- Matriz A = 3000 x 3000 y Matriz B = 3000 x 3000
- Matriz A = 6000 x 6000 y Matriz B = 6000 x 6000
- Matriz A = 6000 x 7000 y Matriz B = 7000 x 6000

La matriz A es una matriz de 0.1 en todas sus posiciones y la matriz B tiene 0.2 en todas. Excepto en el último caso, en el cual ambas matrices tienen números aleatorios en todas sus posiciones. Además, se consideraron diversas cantidades de núcleos para poder realizar un análisis más exhaustivo y variado. Particularmente, estas fueron:

- 1 núcleo
- 2 núcleos
- 4 núcleos
- 8 núcleos
- 16 núcleos
- 32 núcleos

Los resultados obtenidos para cada combinación se presentarán en forma de tabla, el cual contiene el tiempo de ejecución de cada uno, junto a su respectivo speedup

y eficiencia. Para poder obtener resultados más precisos, para cada combinación se realizaron 3 ejecuciones, con las que se calculó el tiempo promedio de ejecución.

En el caso del speedup, el cálculo se realizó considerando el tiempo de ejecución de la versión secuencial y el de la versión paralela correspondiente.

Además de los índices mencionados, también se analizó el balanceo de carga y la escalabilidad de cada ejecución. Para ello, se hizo uso de mediciones de tiempos de cómputo y comunicación, tomados en las ejecuciones de la implementación.

## 4. Resultados obtenidos

Se realizaron las ejecuciones correspondientes a las combinaciones mencionadas en la anterior sección. Las implementaciones fueron ejecutadas en el cluster de la Facultad de Ingeniería de la UNCUIYO con 32 nodos físicos. Se presentan los resultados en una tabla diferente para cada caso.

### 4.1. Resultados con matriz 300 x 300

| N.º | Cantidad de nodos | Tiempo de ejecución promedio (s) | Speedup | Eficiencia |
|-----|-------------------|----------------------------------|---------|------------|
| 1   | 1                 | 0.05475                          | 1       | 1          |
| 2   | 2                 | 0.02624                          | 2.09    | 1.045      |
| 3   | 4                 | 0.01495                          | 3.66    | 0.92       |
| 4   | 8                 | 0.05460                          | 1.0027  | 0.13       |
| 5   | 16                | 0.12281                          | 0.45    | 0.028      |
| 6   | 32                | 0.2301                           | 0.24    | 0.0075     |

Tabla 1: Resultados obtenidos para matrices de tamaño 300 x 300

### 4.2. Resultados con matriz 1000 x 1000

| N.º | Cantidad de nodos | Tiempo de ejecución promedio (s) | Speedup | Eficiencia |
|-----|-------------------|----------------------------------|---------|------------|
| 1   | 1                 | 2.1836                           | 1       | 1          |
| 2   | 2                 | 1.0836                           | 2.0151  | 1.0076     |

|   |    |        |        |        |
|---|----|--------|--------|--------|
| 3 | 4  | 0.5625 | 3.882  | 0.9705 |
| 4 | 8  | 0.4947 | 4.414  | 0.5176 |
| 5 | 16 | 0.5041 | 4.33   | 0.271  |
| 6 | 32 | 0.5259 | 4.1521 | 0.1298 |

Tabla 2: Resultados obtenidos para matrices de tamaño 1000 x 1000

#### 4.3. Resultados con matriz 3000 x 3000

| N.º | Cantidad de nodos | Tiempo de ejecución promedio (s) | Speedup | Eficiencia |
|-----|-------------------|----------------------------------|---------|------------|
| 1   | 1                 | 88.1337                          | 1       | 1          |
| 2   | 2                 | 45.7410                          | 1.93    | 0.97       |
| 3   | 4                 | 23.3967                          | 3.77    | 0.94       |
| 4   | 8                 | 13.5283                          | 6.51    | 0.81       |
| 5   | 16                | 9.8457                           | 8.95    | 0.56       |
| 6   | 32                | 6.2613                           | 14.08   | 0.44       |

Tabla 3: Resultados obtenidos para matrices de tamaño 3000 x 3000

#### 4.4. Resultados con matriz 6000x6000

| N.º | Cantidad de nodos | Tiempo de ejecución promedio (s) | Speedup | Eficiencia |
|-----|-------------------|----------------------------------|---------|------------|
| 1   | 1                 | 610.7383                         | 1       | 1          |
| 2   | 2                 | 305.34                           | 2       | 1          |
| 3   | 4                 | 158.253                          | 3.86    | 0.96       |
| 4   | 8                 | 122.34                           | 4.99    | 0.62       |
| 5   | 16                | 65.60                            | 9.31    | 0.58       |
| 6   | 32                | 33.53                            | 18.21   | 0.57       |

Tabla 4: Resultados obtenidos para matrices de tamaño 6000x6000

#### 4.5. Resultados con matrices de 6000x7000 y 7000x6000

| N.º | Cantidad de nodos | Tiempo de ejecución promedio (s) | Speedup | Eficiencia |
|-----|-------------------|----------------------------------|---------|------------|
| 1   | 1                 | 703.495                          | 1       | 1          |
| 2   | 2                 | 359.855                          | 1.95    | 0.98       |
| 3   | 4                 | 188.946                          | 3.72    | 0.93       |
| 4   | 8                 | 149.707                          | 4.70    | 0.59       |
| 5   | 16                | 79.1322                          | 8.89    | 0.56       |
| 6   | 32                | 49.6141                          | 14.18   | 0.44       |

Tabla 5: Resultados obtenidos para matrices de tamaño 6000x7000 y 7000x6000 con valores aleatorios

Para poder analizar el balanceo de carga, se midieron los tiempos de cómputo de cada nodo y los tiempos de comunicación correspondientes para 3 ejecuciones con 32, 16 y 8 procesos (1 por nodo) con matrices de 6000 x 6000. Posteriormente, se calculó el promedio de las medidas de dichas ejecuciones y se calculó la varianza y desviación estándar de cada uno. Estos resultados se presentan en las siguientes tablas.

#### 4.6. Resultados de tiempos de cómputo

| Cantidad de nodos | Tiempo de cómputo promedio | Varianza de tiempos de cómputo | Desviación estándar de tiempos de cómputo |
|-------------------|----------------------------|--------------------------------|-------------------------------------------|
| 32                | 27.326                     | 8.22                           | 2.87                                      |
| 16                | 51.79                      | 25.71                          | 5.07                                      |
| 8                 | 97.84                      | 337.61                         | 18.37                                     |

Tabla 6: Resultados obtenidos midiendo el tiempo de cómputo por nodo



#### 4.7. Resultados de tiempos de comunicaciones

| Cantidad de nodos | Tiempo de comunicación promedio | Varianza de tiempos de comunicación | Desviación estándar de tiempos de comunicación |
|-------------------|---------------------------------|-------------------------------------|------------------------------------------------|
| 32                | 6.69                            | 6.45                                | 2.54                                           |
| 16                | 12.38                           | 39.01                               | 6.24                                           |
| 8                 | 10.39                           | 153.54                              | 12.39                                          |

Tabla 7: Resultados obtenidos midiendo el tiempo de comunicación por nodo

Además, para poder realizar un análisis más exhaustivo, se presentan los resultados obtenidos por proceso cuando se utilizaron 16 nodos. Es decir, se muestra el tiempo de comunicación y cómputo de cada proceso que formó parte de esta ejecución.

#### 4.8. Tiempos de cómputo y comunicación por proceso para la ejecución con 16 nodos

| Proceso | Tiempo de cómputo promedio (s) | Tiempo de comunicación promedio (s) |
|---------|--------------------------------|-------------------------------------|
| 1       | 50.56                          | 17.40                               |
| 2       | 47.53                          | 5.99                                |
| 3       | 51.19                          | 3.03                                |
| 4       | 46.96                          | 7.27                                |
| 5       | 57.83                          | 7.17                                |
| 6       | 58.13                          | 7.45                                |
| 7       | 56.19                          | 9.46                                |
| 8       | 58.23                          | 7.49                                |
| 9       | 55.06                          | 10.68                               |
| 10      | 49.22                          | 16.53                               |
| 11      | 48.07                          | 17.69                               |

|    |       |       |
|----|-------|-------|
| 12 | 44.82 | 20.94 |
| 13 | 60.78 | 7.16  |
| 14 | 47.29 | 20.66 |
| 15 | 48.82 | 19.14 |
| 16 | 47.91 | 20.05 |

Tabla 8: Resultados obtenidos midiendo el tiempo de cómputo y comunicación por proceso para la ejecución de 6000 x 6000 con 16 procesos

## 5. Análisis de resultados

A partir de los resultados obtenidos, presentados en la anterior sección, se realizó un análisis, principalmente teniendo en cuenta el speedup, la eficiencia, el balanceo de carga y la escalabilidad. Para una mayor claridad, cada aspecto fue separado en una sección diferente, sin embargo, es importante evaluar los 4 índices en conjunto, ya que cada uno nos permite observar una parte diferente del rendimiento del programa paralelo.

### 5.1. Speedup

El speedup nos permite identificar cuánto mejor es un programa paralelo con respecto a su versión secuencial. Por este motivo, lo analizaremos para los distintos tamaños de matrices considerados en el diseño de experimentos. El caso de 1000 x 1000 no fue incluido debido a su similitud con el caso de 300 x 300

#### 5.1.1. 300 x 300

Se puede observar que el speedup comienza creciendo hasta los 4 nodos, sin embargo, a partir de los 8, decae notablemente, siendo esta ejecución sólo ligeramente mejor que la secuencial. Si se ve la ejecución con 16 y 32 nodos, se puede notar que esta es incluso peor que la secuencial, probablemente debido a que la cantidad de comunicaciones degrada el rendimiento, ya que el pequeño tamaño del problema no justifica la cantidad de comunicaciones existentes. Teniendo esto en cuenta, podemos concluir que, al menos con una matriz de pequeña dimensión, la ganancia de utilizar varios nodos es muy poca debido a las comunicaciones, y, en el caso de ser necesario, se debería utilizar máximo 4 nodos, pues si no el rendimiento sería peor que el secuencial y los procesadores adicionales serían desperdiciados.

Otro aspecto importante a notar es que con 2 nodos, el speedup es superlineal. Esto probablemente debido a que se aprovecha la memoria adicional para realizar la multiplicación, lo que permite obtener un tiempo de ejecución mejor que el esperado. Sin embargo, con más nodos

esta situación no se da debido a que la memoria adicional que nos brindan el resto de los nodos ya no es tan necesaria y no se aprovecha tanto como en el caso de los 2 nodos.

### **5.1.2. 3000 x 3000**

A partir de este caso, se observa una situación diferente con respecto al speedup. Se ve que para todos los nodos este aumenta, en ningún momento decrece. Además, se observa en la figura 1 que hasta los 4 nodos, este crece casi de manera lineal, es decir, acercándose a la situación ideal, sin embargo, luego de los 4 empieza a crecer más lentamente. Esta situación probablemente es debido a que comienzan a haber muchas comunicaciones, lo que provoca que el nodo 0 sea un cuello de botella y que, al demorarse más la transferencia de datos, el rendimiento del resto de nodos también se vea degradado.

Además del impacto de las comunicaciones, también influye un límite teórico del paralelismo: la Ley de Amdahl. Esto indica que, por más nodos que se agreguen, el rendimiento no puede mejorar indefinidamente, ya que la porción secuencial del programa (como la inicialización, y la comunicación) permanece constante. Entonces, para que sea beneficioso paralelizar, la fracción paralela debe ser mucho mayor que la secuencial. En nuestro caso, aunque la multiplicación de matrices es altamente paralelizable, al dividir el trabajo entre más nodos, la parte paralela asignada a cada uno se reduce, y la parte secuencial gana peso relativo, afectando negativamente la aceleración total.

Sin embargo, debido a que el speedup crece en todo momento, en el caso de que estuviéramos en un contexto en el que la velocidad sea crítica o muy importante, nos convendría utilizar el máximo número de nodos disponibles para obtener esa ganancia adicional en la velocidad de ejecución.

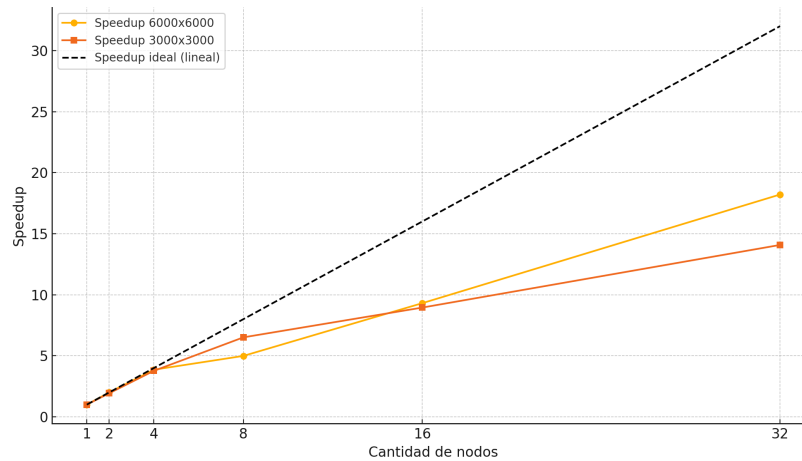


Figura 1: Speedup para tamaños de 6000 x 6000 y 3000 x 3000

### 5.1.3. 6000 x 6000

Este caso obtuvo un conjunto de valores que posee un comportamiento muy similar al caso anterior. Es decir, hasta los 4 nodos el speedup crece de manera casi ideal, sin embargo, luego empieza a crecer más lentamente. Además, se observa en la figura 1 que a partir de los 16 nodos, con este tamaño del problema se obtiene un speedup más alto que en el caso anterior, donde la dimensión era menor.

Nuevamente, si se estuviese en una situación en la que se necesite obtener los resultados rápidamente, se justificaría totalmente el uso de una gran cantidad de procesadores.

### 5.1.4. 6000 x 7000 y 7000 x 6000

En este caso, al igual que en el anterior, se observa un comportamiento de speedup creciente a medida que se incrementa la cantidad de nodos. Hasta los 4 nodos, el crecimiento es casi lineal, lo que indica una buena paralelización inicial del problema. Sin embargo, a partir de los 8 nodos el incremento del speedup comienza a desacelerarse, alcanzando un valor de 14.18 con 32 nodos. Esto indica que, si bien el paralelismo sigue aportando mejoras en tiempo de ejecución, los beneficios adicionales crecen más lentamente con más nodos debido al aumento relativo en las comunicaciones y sincronizaciones necesarias.

En comparación con el caso de matrices cuadradas de tamaño 6000, se observa que este último presenta valores de speedup consistentemente mayores. Sin embargo, es importante tener en cuenta que las matrices no cuadradas implican una mayor cantidad de operaciones, dado que una de sus dimensiones es de 7000. Además, rellenar las matrices con números

aleatorios implica un tiempo mayor que en el resto de los casos. Por ello, la parte secuencial de la ejecución aumenta y, según la ley de Amdahl, esto influye en el speedup y podría ser otra de las razones por las que el speedup es menor al de la anterior ejecución. Por lo tanto, considerando estas diferencias en el tamaño del problema, podemos concluir que el uso de matrices no cuadradas con valores aleatorios afecta, pero no significativamente el comportamiento del speedup, ya que se mantiene un crecimiento similar al observado en el caso cuadrado.

## 5.2. Eficiencia

La eficiencia nos permite identificar el uso que se ha hecho de los procesadores. Por ello, como al speedup, lo analizaremos teniendo en cuenta cada tamaño considerado. Nuevamente, el caso de 1000 x 1000 no fue incluido debido a su similitud con el caso de 300 x 300

### 5.2.1. 300 x 300

Se observa que hasta 4 nodos, la eficiencia es muy buena, incluso con 2 nodos es mayor a 1, esto debido a que se tiene un speedup superlineal en este punto. Sin embargo, desde los 8 nodos en adelante, la eficiencia decae estrepitosamente, llegando al punto de ser de solo 0.75% con 32 procesadores. De esto se concluye que para matrices de pequeño tamaño, el agregado de más de 4 nodos no resulta beneficioso ya que casi no se aprovechan los procesadores adicionales.

Si se quisiera ahorrar en recursos, pero obteniendo buen rendimiento, se debería optar por elegir 4 nodos, pues se obtiene el máximo speedup posible con una eficiencia del 92%, la cual es casi perfecta.

### 5.2.2. 3000 x 3000

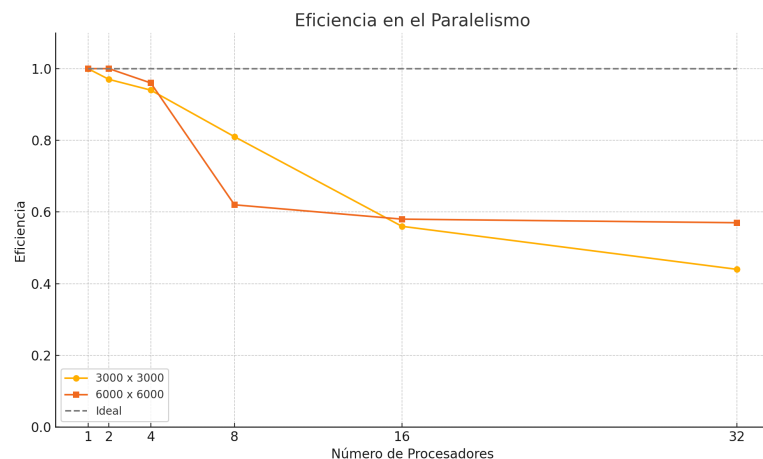


Figura 2: Eficiencia para tamaños de 3000 x 3000 y 6000 x 6000

Se observa en la figura 2 que la eficiencia decae más lentamente que en los casos anteriores, comenzando con una eficiencia muy buena. A los 16 nodos ya es del 56%, lo que podría ser aceptable si se necesitara una buena velocidad.

Para esta situación, si se necesitara velocidad, se podría considerar elegir 16 nodos, pues con esta cantidad se posee un gran speedup y una eficiencia media, lo cual nos indicaría que se desaprovechan recursos, pero no en un porcentaje desmesurado. En caso de que se necesite priorizar el ahorro de recursos, podría optarse por 8 nodos, que nos da una eficiencia muy buena, del 81%, y un speedup beneficioso.

#### **5.2.3. 6000 x 6000**

Se tiene una situación similar a la anterior, sin embargo, a los 8 nodos, la eficiencia cae con mayor velocidad que en el caso de las matrices de 3000 x 3000. Además, a partir de los 8 procesadores, la eficiencia va decayendo, pero muy lentamente. Esto nos podría indicar que, mientras mayor el tamaño del problema, se desaprovecharían menos los nodos con respecto a dimensiones menores.

#### **5.2.4. 6000 x 7000 y 7000 x 6000**

La eficiencia en este caso muestra un patrón de descenso conforme se incrementa la cantidad de nodos. Si bien comienza en 1 con un solo nodo y se mantiene alta hasta los 4 nodos (0.93), a partir de los 8 nodos la caída es más pronunciada, llegando a un valor de 0.44 con 32 nodos. Esta disminución sugiere que los tiempos de comunicación y sincronización entre nodos comienzan a representar una porción significativa del tiempo total, reduciendo el aprovechamiento efectivo de los recursos. A pesar de esto, el uso de múltiples nodos sigue siendo ventajoso si se prioriza la reducción del tiempo de ejecución por sobre la eficiencia total del sistema.

### **5.3. Balanceo de carga**

El balanceo de carga nos indica si se ha repartido equitativamente el trabajo entre los nodos con el objetivo de minimizar el tiempo ocioso de estos.

Teniendo en cuenta la desviación estándar, podemos ver que mientras menos nodos se utilizan, más se desvían los tiempos de cómputo de la media. Lo mismo sucede con los tiempos de comunicación. Esto nos indicaría que mientras mayor cantidad de nodos, mejor se distribuye la carga del trabajo. Además, esto se explicaría por la memoria, es decir, al tener menos nodos, cada uno de estos recibe mayor cantidad de datos para computar, y si estos tienen una memoria limitada, demorarán más tiempo realizando el cómputo. Lo mismo sucede con el tiempo de

comunicación, si se observa la cantidad de tiempo que el proceso 0 pasa comunicando con 8 nodos, esta es de 41 segundos, frente a 17 segundos con 16 nodos o 12 segundos con 32 procesadores. Esto indicaría que resulta más eficiente comunicar una mayor cantidad de datos pequeños que una menor cantidad de datos más grandes. Es decir, considerando una granularidad más chica se tiene un mejor balanceo de carga ya que los procesadores pasan menos tiempo comunicando y, en promedio, todos pasan una cantidad de tiempo similar computando.

Además, se mostraron los resultados individuales de cada proceso para la ejecución con 16 nodos y con dos matrices de 6000 x 6000. Se puede observar que entre el mejor y el peor tiempo de cómputo hay una diferencia del 35%, lo cual es una diferencia bastante notoria. Además, si se tiene en cuenta el tiempo de comunicación, se observa que el peor es de 20.94 segundos frente al mejor de 3.03 segundos. Es decir, hay una enorme diferencia entre los distintos procesos. Sin embargo, estos dos tiempos están relacionados, ya que, por ejemplo, el que tuvo el peor tiempo de comunicación, tuvo el mejor tiempo de cómputo. Esto indicaría que este proceso (el 12), realizó tan rápido su cómputo que tuvo mucho tiempo ocioso esperando a poder enviar sus resultados al proceso 0. Esta diferencia en los tiempos de cómputo puede deberse a varios factores, entre ellos: posibles diferencias en las velocidades de los nodos del cluster; uso compartido de núcleos físicos debido a la ejecución dentro del cluster de la facultad, latencias propias de la red; entre otros.

En conclusión, los datos analizados presentaron un desbalance sustancial de los tiempos de comunicación y de cómputo entre cada proceso. Además, se observó que el balanceo de carga es mejor a medida que se aumenta la cantidad de nodos.

#### **5.4. Escalabilidad**

La escalabilidad menciona que si aumentamos la cantidad de recursos computacionales, dejando el tamaño del problema fijo, el tiempo de ejecución disminuirá. Otra definición válida es que si escalamos el tamaño del problema y la cantidad de nodos proporcionalmente, el tiempo de ejecución se mantendrá constante.

En el caso del programa realizado, podemos ver que sí es escalable, puesto que en la mayoría de los casos, exceptuando la multiplicación de matrices muy pequeñas, si mantenemos fijo el tamaño del problema y aumentamos los nodos, el tiempo de ejecución disminuye. Esto se puede corroborar observando el speedup. Si este es cada vez mayor, significa que el tiempo de ejecución es cada vez menor.

Sin embargo, la escalabilidad de la implementación no es óptima, puesto que si, por ejemplo, se duplica la cantidad de nodos manteniendo fijo el tamaño del problema, el tiempo de ejecución no se reducirá a la mitad. Por otro lado, esto es

muy difícil de alcanzar y teóricamente imposible, pues mientras más nodos se agregan, más comunicaciones hay que gestionar, lo que puede ocasionar saturaciones y tiempos de transmisión mayores. Además, teniendo en cuenta la ley de Amdahl, la región secuencial de la implementación es otro factor que limita el paralelismo que es posible alcanzar.

## 6. Conclusiones

En conclusión, se identificó que el rendimiento del programa paralelo depende fuertemente del tamaño del problema y del número de procesadores utilizados. Para matrices pequeñas, el uso de excesiva cantidad de nodos puede generar *overhead* y tener peor rendimiento que en el caso del secuencial debido a los altos costos de comunicación. En estos casos se observó que lo más eficiente es utilizar una cantidad reducida de procesadores.

En cambio, para problemas de mayor tamaño, se observa un speedup creciente con mayor cantidad de procesadores y un mejor balanceo de carga, lo que justifica el uso de más nodos si la prioridad es la velocidad. Sin embargo, la eficiencia disminuye progresivamente al aumentar los nodos, por lo que se debe tener en cuenta el *trade-off* entre velocidad y uso de recursos.

El programa es escalable para tamaños grandes, pero está limitado por la comunicación y las partes secuenciales del código. Sin embargo, en general la versión paralela es efectiva si se tiene en cuenta la escala del problema.

Por otro lado, posibles mejoras para la versión paralela podrían ser: que solo el proceso 0 calcule y envíe los vectores iniciales mediante broadcast en vez de que lo calculen todos los nodos, para evitar cómputo innecesario; evaluar que el proceso 0 se ocupe de las comunicaciones y del manejo de las filas sobrantes de la división para así reducir las comunicaciones asociadas y evitar el cálculo de vectores iniciales; y considerar implementar el algoritmo de Cannon, el cual cuenta con referencias en portales de documentación y enciclopedias generales, y cuyo objetivo es minimizar las comunicaciones entre nodos, lo cual podría ser más efectivo en los resultados de la paralelización.

## 7. Referencias bibliográficas

[LAY14] Lay, David C.: Linear Algebra and Its Applications. Fifth edition, Pearson, 2014: pp. 96-100.

[MPI25] Open MPI: Open MPI v5.0.x Documentation.

<https://docs.open-mpi.org/en/v5.0.x/> (Accedido 10 de Mayo de 2025)

[PPYD25] Caymes Scutari, Paola: Apuntes de Programación Paralela y Distribuida. 2025



[CANN25] Wikipedia: Algoritmo de Cannon.  
[https://en.wikipedia.org/wiki/Cannon%27s\\_algorithm](https://en.wikipedia.org/wiki/Cannon%27s_algorithm) (Accedido el 10 de Mayo de 2025)